Trevor Labat
Student ID: 801340975
Github: https://github.com/TlabatCharlotte/Machine-Learning

Homework #6

Problem 1 - Fully Connected Neural Network for Housing Dataset

Using train_test_split method from sklearn.model_selection the housing data is split into a training set 80% and 20% test set. Standardization is immediately applied afterwards using the sklearn.preprocessing library. The neural network is built and trained using PyTorch.

Assumptions made: The StandardScaler is fit on the training data only and then applied to the test data. This is to prevent information leakage. Random_state = 100 (same as HW 2 and HW 4 for the same data split). The yes/no values were mapped to ones and zeros. The furnishingstatus column was dropped so the network uses the same 11 features as HW 2 and HW 4, which keeps the comparison in part b fair. Price was also standardized for training and converted back to dollars using inverse_transform before the metrics were calculated. The 20% test set is used to plot the validation loss while training, but backward is only called on the training loss so the model never trains on the test data. The whole training set is used every epoch (full batch).

The network is built with nn.Sequential and has two hidden layers. The code for the network and optimizer can be seen in Figure 1.1.

```
seq_model = nn.Sequential(OrderedDict([('hidden_linear1', nn.Linear(11,8)),
                                       ('hidden_activation1', nn.Tanh()),
                                       ('hidden_linear2', nn.Linear(8,8)),
                                       ('hidden_activation2', nn.Tanh()),
                                       ('output_linear', nn.Linear(8,1))]))

optimizer = optim.SGD(seq_model.parameters(), lr=0.01)
```

Figure 1.1 - Housing neural network architecture

The architecture and hyperparameters used are the following:

- Architecture
  - Input layer: 11 features
  - Hidden layer 1: 8 neurons with Tanh activation
  - Hidden layer 2: 8 neurons with Tanh activation
  - Output layer: 1 neuron (the predicted price) with no activation since this is regression
  - Total parameters: 177
- Hyperparameters
  - Optimizer: SGD
  - Learning rate: 0.01
  - Epochs: 3000
  - Loss function: MSE (nn.MSELoss)
  - Batch size: full batch (all 436 training houses each epoch)

A small network with 8 neurons per layer was used since the dataset only has 545 houses. A bigger network would have a lot more parameters than houses to learn from and would be more likely to overfit. Tanh was used for the activation since it was the activation used in lecture, and it lets the network learn non-linear patterns that linear regression can't. A learning rate of 0.01 is the same value used in previous homeworks.

The training loop is the same one from lecture. The training and validation loss are saved every epoch so they can be plotted, which can be seen in Figure 1.2.

```
def training_loop(n_epochs, optimizer, model, loss_fn, X_train, Y_train, X_test, Y_test):

  train_losses = []
  test_losses = []

  for epoch in range(1, n_epochs + 1):
    prediction = model(X_train)
    loss_train = loss_fn(prediction, Y_train)
    validation = model(X_test)
    loss_test = loss_fn(validation, Y_test)

    optimizer.zero_grad()
    loss_train.backward()
    optimizer.step()

    train_losses.append(loss_train.item())
    test_losses.append(loss_test.item())

    if epoch == 1 or epoch % 1000 == 0:
      print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},"
            f"Validation loss {loss_test.item():.4f}")

  return train_losses, test_losses
```

Figure 1.2 - Training loop

The training and validation loss (scaled) are the following:

- Epoch 1: training loss 0.9172, validation loss 0.8825
- Epoch 1000: training loss 0.3399, validation loss 0.3130
- Epoch 2000: training loss 0.3184, validation loss 0.2981
- Epoch 3000: training loss 0.3045, validation loss 0.2900

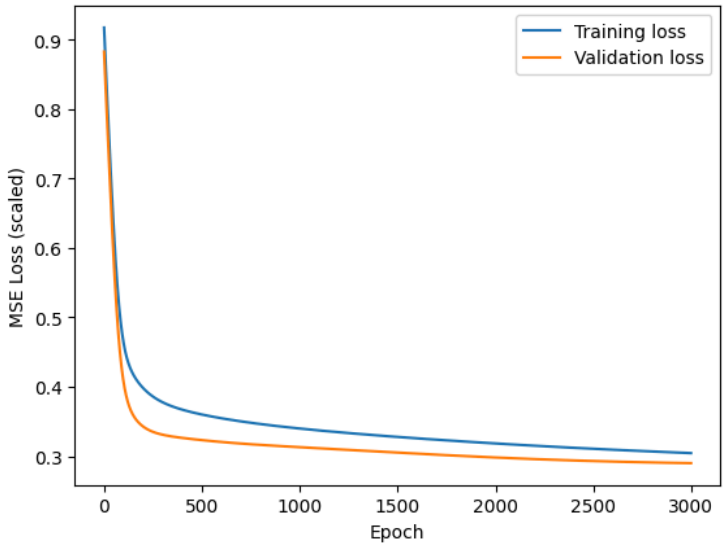


Figure 1.3 - Housing training and validation loss over epochs

Both losses drop quickly in the first 200 epochs and then slowly level off. The validation loss stays below the training loss the entire time, so there are no signs of overfitting. The losses were still going down slowly at epoch 3000, so more epochs may improve the model a little.

The sklearn metrics library is used to calculate the MSE and R2 on the test set after the predictions are converted back to dollars. The root of the MSE was taken to get the error in dollars. The final results are the following:

- MSE: 1.00874×10^12
- RMSE: $1,004,362
- R2: 0.71419

The neural network is compared with the regularized linear regression from HW 2 (alpha = 0.01, lambda = 0.001, 2000 iterations) and the SVR from HW 4. The linear kernel is used for the SVR since it was the best kernel in HW 4. All three models use random_state = 100, the same 11 features, and the same test houses. The test results are the following:

- Neural network
    - MSE: 1.00874×10^12
    - RMSE: 1004362
    - R2: 0.71419
- SVR (linear kernel, HW 4)
    - MSE: 1.14629×10^12
    - RMSE: 1070650
    - R2: 0.67522
- Linear regression (HW 2)
    - MSE: 1.24984×10^12
    - RMSE: 1117970
    - R2: 0.64589

Looking at these results the neural network is the best model and it does outperform the other two. It has the lowest RMSE at $1.00M and the highest R2 at 0.714. Its RMSE is about $66k lower than the SVR and about $114k lower than linear regression. This may be attributed to the Tanh activations in the hidden layers. Linear regression and the linear SVR can only fit a straight line through the data, while the neural network can bend to fit non-linear patterns between the features and price.

The improvement is not huge, which shows the housing data is mostly linear. Also the SVR parameters were not tuned in HW 4 and HW 2 may not have fully converged, so the comparison is not totally fair. An R2 of 0.714 also means about 29% of the change in price still isn't explained by any of the models, which may be from things that aren't in the dataset.

Problem 2 - Fully Connected Neural Network for Cancer Dataset

Using train_test_split method from sklearn.model_selection the cancer data is split into a training set 80% and 20% test set. Standardization is immediately applied afterwards using the sklearn.preprocessing library. The neural network is built and trained using PyTorch.

Assumptions made: The StandardScaler is fit on the training data only and then applied to the test data to prevent information leakage. Random_state = 4 (same as HW 3 and HW 4 for the same data split). Label mapping: M = 1, B = 0. All 30 features were used and the id column was skipped. Classification with neural networks was not covered in lecture, so I used AI to learn how and discovered BCEWithLogitsLoss. The output layer gives a raw value and BCEWithLogitsLoss applies the sigmoid inside the loss. For predictions the sigmoid is applied with a 0.5 threshold, the same threshold used in HW 3. Like Problem 1, the test set is used to plot the validation curves but the model never trains on it, and full batch training is used.

2.a

The network is built the same way as Problem 1. The code for the network and optimizer can be seen in Figure 2.1.

```
seq_model = nn.Sequential(OrderedDict([('hidden_linear1', nn.Linear(30,40)),
                                       ('hidden_activation1', nn.Tanh()),
                                       ('hidden_linear2', nn.Linear(40,40)),
                                       ('hidden_activation2', nn.Tanh()),
                                       ('output_linear', nn.Linear(40,1))]))

optimizer = optim.SGD(seq_model.parameters(), lr=0.01)
```

Figure 2.1 - Cancer neural network architecture

The architecture and hyperparameters used are the following:

- Architecture
  - Input layer: 30 features
  - Hidden layer 1: 40 neurons with Tanh activation
  - Hidden layer 2: 40 neurons with Tanh activation
  - Output layer: 1 neuron (malignant or benign)
  - Total parameters: 2921
- Hyperparameters
  - Optimizer: SGD
  - Learning rate: 0.01
  - Epochs: 3000
  - Loss function: binary cross entropy (nn.BCEWithLogitsLoss)
  - Batch size: full batch (all 455 training samples each epoch)

The hidden layers are bigger than Problem 1 since there are almost three times as many input features. Tanh was used again for the same reason as Problem 1. Binary cross entropy is used

instead of MSE since this is a classification problem, which matches the logistic regression cost function from HW 3.

The training loop is the same as Problem 1, but the training and validation accuracy are also calculated every epoch so they can be plotted. The added lines can be seen in Figure 2.2.

```
train_acc = ((torch.sigmoid(prediction) >= 0.5).float() == Y_train).float().mean().item()
test_acc = ((torch.sigmoid(validation) >= 0.5).float() == Y_test).float().mean().item()
train_accs.append(train_acc)
test_accs.append(test_acc)
```

Figure 2.2 - Accuracy added to the training loop

The training and validation loss are the following:

- Epoch 1: training loss 0.6889, validation loss 0.7107
- Epoch 1000: training loss 0.0706, validation loss 0.0803
- Epoch 2000: training loss 0.0576, validation loss 0.0696
- Epoch 3000: training loss 0.0528, validation loss 0.0674

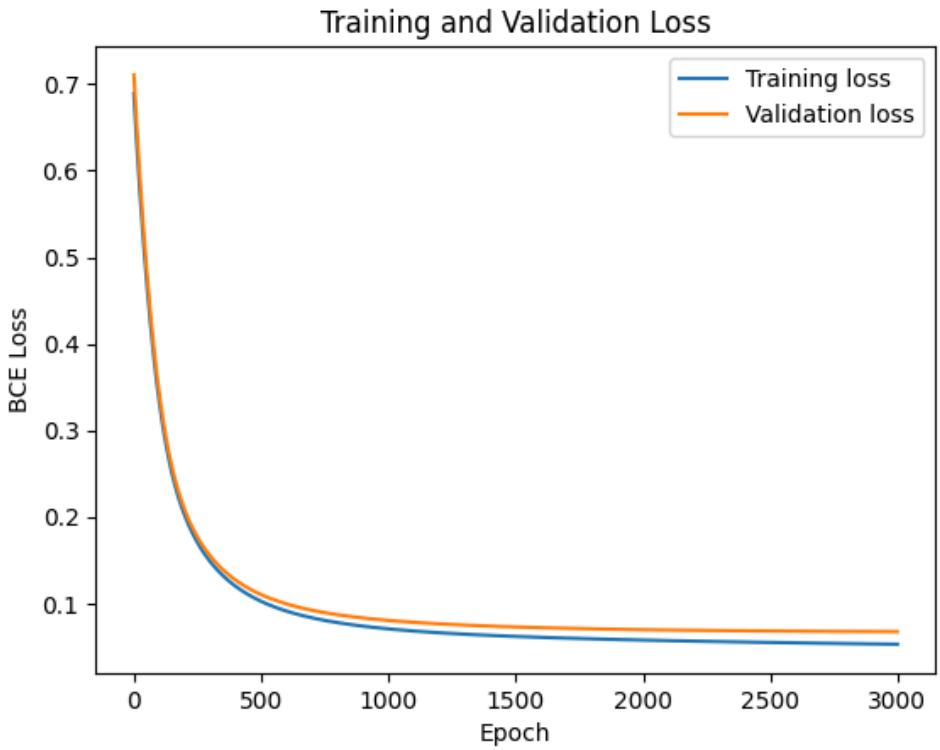


Figure 2.3 - Cancer training and validation loss over epochs

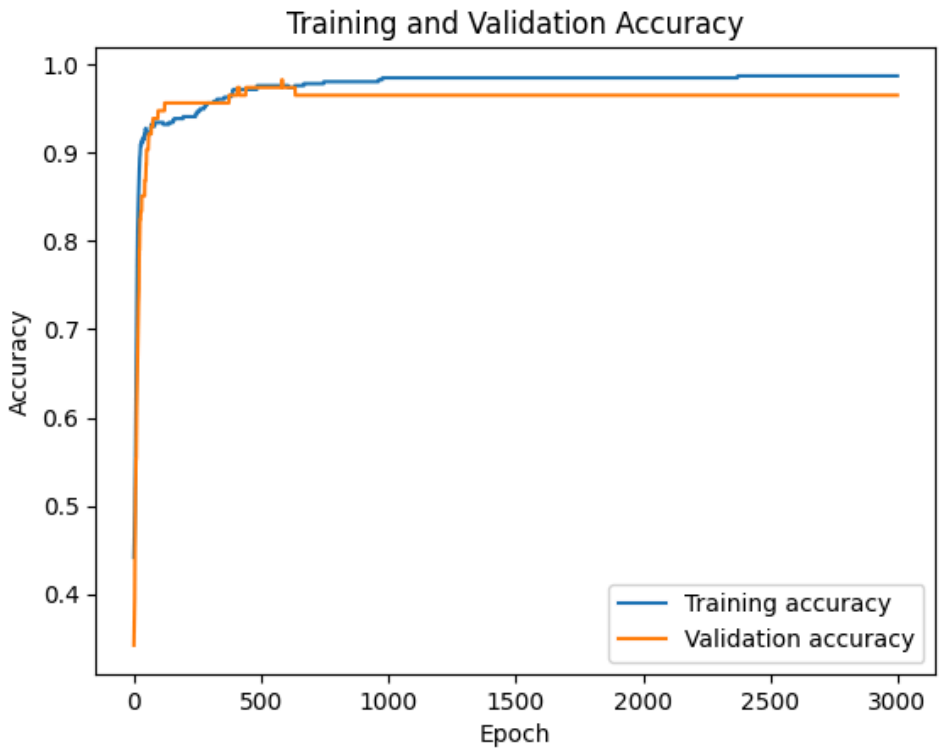


Figure 2.4 - Cancer training and validation accuracy over epochs

The loss for both training and validation drop together and level off around 1500 epochs, with the validation loss staying a little above the training loss. Looking at the accuracy plot, the validation accuracy peaks around 0.98 at roughly 600 epochs and then drops to 0.965 and stays there, while the training accuracy keeps climbing to almost 0.99. This shows signs of mild overfitting, similar to the non-regularized model in HW 3.

The sklearn metrics library is used for calculating the accuracy, precision, recall, and F1 score on the test set. The final values are the following:

- Accuracy: 0.96491
- Precision: 0.89474
- Recall: 1.0
- F1: 0.94444

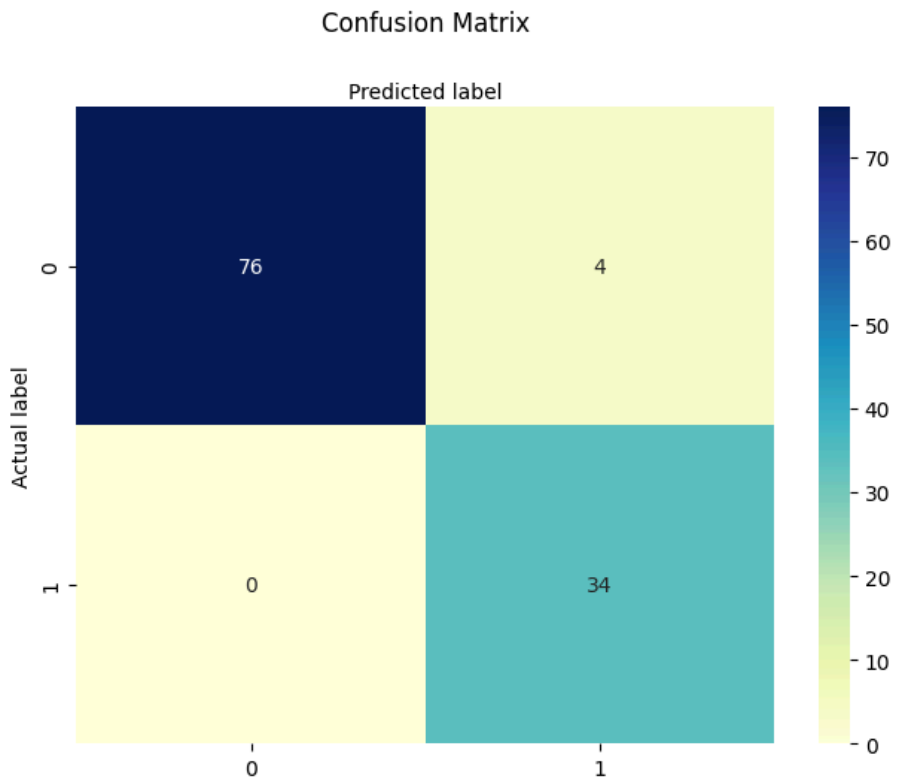


Figure 2.5 - Cancer neural network confusion matrix

The confusion matrix in Figure 2.5 shows 76 true negatives, 4 false positives, 0 false negatives, and 34 true positives. All 34 malignant tumors were caught, which is why the recall is 1.0. The only mistakes were 4 benign tumors labeled as malignant, which is what lowers the precision.

2.b

The neural network is compared with the regularized logistic regression from HW 3 (lambda = 10) and the SVM from HW 4. The RBF kernel is used for the SVM since it was the best kernel in HW 4. All three use random_state = 4 and the same test set. The F1 scores for HW 3 and HW 4 were found using the precision and recall from those homeworks. The test results are the following:

- Neural network
  - Accuracy: 0.96491
  - Precision: 0.89474
  - Recall: 1.0
  - F1: 0.94444
- Logistic regression (HW 3, lambda = 10)
  - Accuracy: 0.99123
  - Precision: 0.97143
  - Recall: 1.0
  - F1: 0.98551

- SVM (RBF kernel, HW 4)
  - Accuracy: 0.97368
  - Precision: 0.91892
  - Recall: 1.0
  - F1: 0.95775

Looking at these results the regularized logistic regression from HW 3 is the best model and the neural network does not outperform it. The neural network actually scored the lowest of the three. All three models have a recall of 1.0, so the only difference is the number of false positives: 4 for the neural network, 3 for the SVM, and 1 for logistic regression.

The neural network results are exactly the same as the non-regularized logistic regression from HW 3, which had the same confusion matrix (76, 4, 0, 34) and the same metrics. This means the neural network learned about as well as a basic logistic regression. What gave HW 3 the edge was the regularization, which the neural network did not use. This may be attributed to the cancer data being simple enough that a straight line already separates the two classes well, so the extra layers don't add much. With 2921 parameters and only 455 training samples, the network also starts to overfit as seen in Figure 2.4. Adding regularization or stopping training around 600 epochs may close the gap, but this wasn't tested.

When considering the metrics, recall matters the most because missing a malignant tumor is worse than a false alarm. All three models reached a recall of 1.0, so any of them would be safe to use, but logistic regression had the fewest false alarms.